Машинное обучение

Лекция 2: Линейные классификаторы и персептрон

Дьулустан Никифоров

Кафедра ИТ
Северо-Восточный Федеральный Университет

Весна 2026

Линейные бинарные классификаторы

Будем рассматривать бинарные классификаторы: $y \in \{-1, +1\}$.
Обучающая выборка:

$$\mathcal{D}_n = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$$

Рассматриваем класс гипотез — класс **линейных классификаторов** $\mathcal{H}$:

$$h(x; \theta, \theta_0) = sign(\theta^T x + \theta_0) = \begin{cases} +1 & \text{, если } \theta^T x + \theta_0 > 0 \\ -1 & \text{, иначе} \end{cases}$$

$\theta \in \mathbb{R}^d,\ \theta_0 \in \mathbb{R}$.
$x^{(i)}$ — вектор $d \times 1$
θ — вектор $d \times 1$

Линейные классификаторы

Ошибка обучения (training error) классификатора h:

$$\mathcal{E}_n(h) = \frac{1}{n}\sum_{i=1}^{n}\begin{cases}1, & \text{если } h(x^{(i)}) \neq y^{(i)} \\ 0, & \text{иначе.}\end{cases}$$

Мы хотим из класса гипотез H выбрать ту, которая минимизирует $\mathcal{E}_n(h)$.

Как это сделать — это есть выбор **алгоритма обучения**.

В конце прошлой лекции мы рассмотрели очень простой алгоритм — **Random linear classifier**.

Сейчас рассмотрим алгоритм под названием **Персептрон (perceptron)**, придуманный Фрэнком Розенблаттом в 1957 году.

Алгоритм персептрона

$\textsc{Perceptron}(\tau, \mathcal{D}_n)$

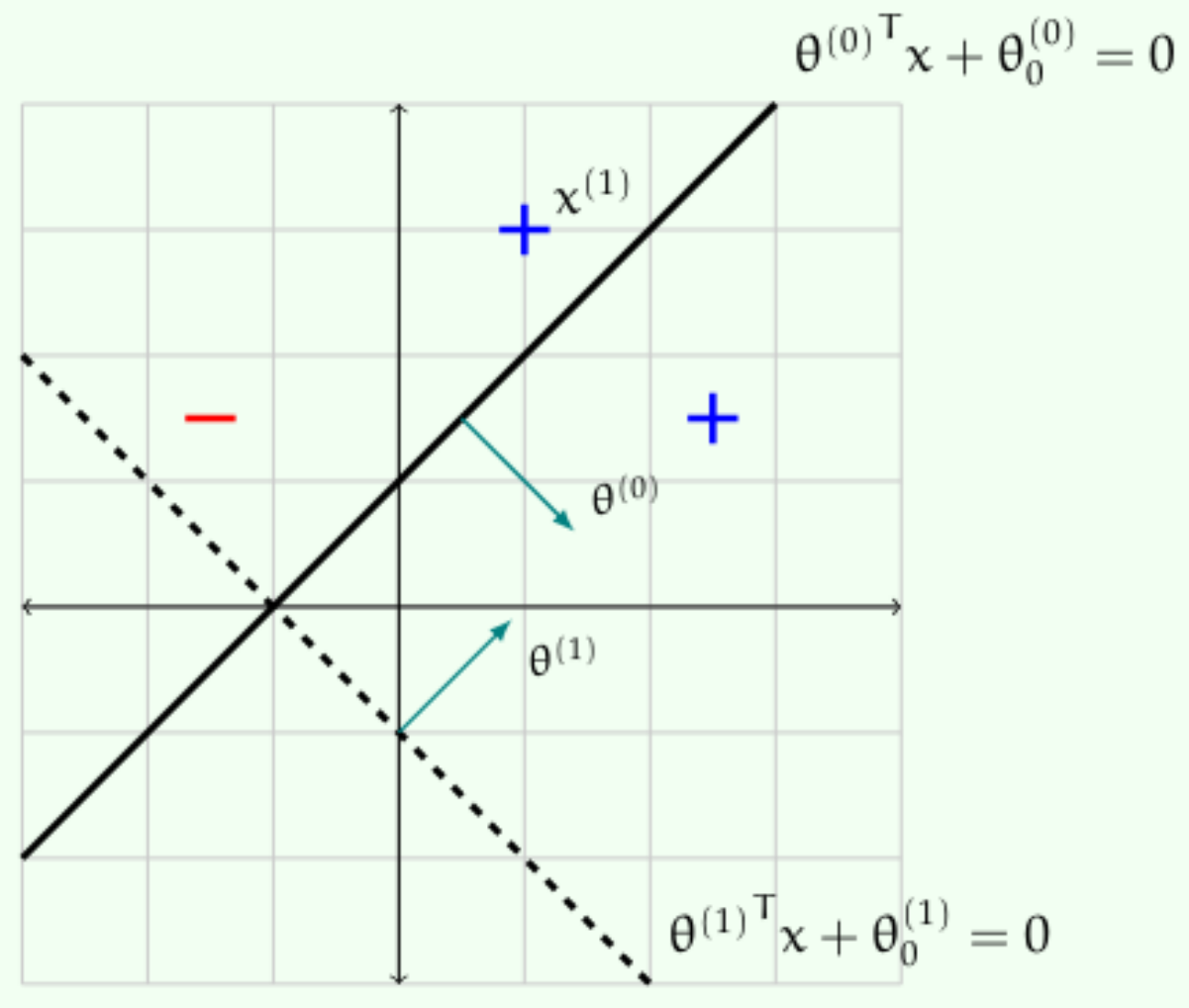
$\theta = \begin{bmatrix} 0 & 0 & \cdots & 0 \end{bmatrix}^T$
$\theta_0 = 0$
for $t = 1$ **to** τ
    for $i = 1$ **to** n
        if $y^{(i)} \left(\theta^T x^{(i)} + \theta_0\right) \leqslant 0$
            $\theta = \theta + y^{(i)} x^{(i)}$
            $\theta_0 = \theta_0 + y^{(i)}$
return θ, θ_0

Алгоритм персептрона

Идея алгоритма:

- $y^{(i)}(\theta^T x^{(i)} + \theta_0) \leqslant 0$ означает, что наша модель неправильно классифицирует объект $(x^{(i)}, y^{(i)})$.
- В этом случае модель «подправляет» себя:
  - Если $y^{(i)} < 0$ и $\theta^T x^{(i)} + \theta_0 \geqslant 0$, то $\theta \to \theta - x^{(i)}$, $\theta_0 \to \theta_0 - 1$.
    Можем заметить, что $\theta^T x^{(i)} + \theta_0$ уменьшается, т.е. становится ближе к «правильному» отрицательному значению.
  - Если $y^{(i)} > 0$ и $\theta^T x^{(i)} + \theta_0 \leqslant 0$, то $\theta \to \theta + x^{(i)}$, $\theta_0 \to \theta_0 + 1$.
    Можем заметить, что $\theta^T x^{(i)} + \theta_0$ увеличивается, т.е. становится ближе к «правильному» положительному значению.

Персептрон: пример одной итерации



Персептрон: пример одной итерации

Модель $\left(\theta^{(0)} = \begin{bmatrix} 1 \\ -1 \end{bmatrix},\ \theta_0^{(0)} = 1\right)$ ошибочно классифицирует точку $x^{(1)} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$ с меткой $y^{(1)} = 1$:

$y^{(1)}({\theta^{(0)}}^T x + \theta^{(0)}) = -1 < 0.$

Соответственно алгоритм персептрона делает подправку:

$\theta^{(1)} = \theta^{(0)} + y^{(1)} x^{(1)} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$

$\theta_0^{(1)} = \theta_0^{(0)} + y^{(1)} = 2.$

Новая модель $(\theta^{(1)},\ \theta_1^{(1)})$ теперь правильно классифицирует точку $x^{(1)}$, но стало ошибаться на другой точке!

Оффсет

Мы везде таскаем свободный коэффициент — **оффсет (offset)** θ_0.
А давайте просто уберем её из наших моделей:

$$h(x;\theta) = sign(\theta^T x) = \begin{cases} +1 & \text{, если } \theta^T x > 0 \\ -1 & \text{, иначе} \end{cases}$$

Теперь все наши *разделяющие гиперплоскости* проходят через начало координат.
Имеем ли мы право рассматривать только такие модели?

Оффсет

Да! Для этого надо к каждой точке добавить новую размерность с координатой $+1$:

$$\bar{x} = \begin{bmatrix} x_1 & \dots & x_d & +1 \end{bmatrix}$$
$$\bar{\theta} = \begin{bmatrix} \theta_1 & \dots & \theta_d & \theta_0 \end{bmatrix}$$

Заметим, что

$$\bar{\theta}^T \bar{x} = \theta_1 x_1 + \dots + \theta_d x_d + \theta_0 \cdot 1 = \theta^T x + \theta_0$$

Значит, мы можем при желании перейти от d-мерных линейных классификаторов с оффсетом к эквивалентным $(d+1)$-мерным линейным классификаторам без оффсета.

Алгоритм персептрона через начало

Используя вышеуказанный метод, можно предыдущий алгоритм персептрона свести к чуть более простому алгоритму **персептрона через начало**:

$\textsc{Perceptron-Through-Origin}(\tau, \mathcal{D}_n)$

$\theta = \begin{bmatrix} 0 & 0 & \cdots & 0 \end{bmatrix}^T$
for $t = 1$ **to** τ
  for $i = 1$ **to** n
    if $y^{(i)} \left(\theta^T x^{(i)}\right) \leqslant 0$
      $\theta = \theta + y^{(i)} x^{(i)}$
return θ

Линейная разделяемость

Definition

Обучающая выборка $\mathcal{D}_n$ называется **линейно разделяемой (linearly separable)**, если существуют θ, θ_0 такие, что

$$y^{(i)}(\theta^T x^{(i)} + \theta_0) > 0$$

для всех $i = 1, \ldots, n$.
Альтернативно можно сказать, что ошибка обучения $\mathcal{E}_n(h) = 0$.

Definition

Обучающая выборка $\mathcal{D}_n$ называется **линейно разделяемой через начало (linearly separable through origin)**, если существует θ такое, что

$$y^{(i)}(\theta^T x^{(i)}) > 0$$

для всех $i = 1, \ldots, n$.

Линейная разделяемость

- Обучающая выборка $\mathcal{D}_n$ может быть линейно разделяемой, но не линейно разделяемой через начало.
- От линейно разделяемой $\mathcal{D}_n$ можно перейти к линейно раздяемой через начало $\bar{\mathcal{D}}_n$ добавлением новой размерности.
- Очевидно, линейная разделяемость необходима, если мы хотим достигнуть нулевых ошибок обучения линейного классификатора.

Отступ (margin)

Вспомним, что для гиперплоскости $\theta^T x + \theta_0 = 0$, расстояние (со знаком) от неё до точки x равно

$$\frac{\theta^T x + \theta_0}{\|\theta\|}.$$

Назовём **отступом (margin)** помеченной точки (x, y) по отношению к гиперплоскости (θ, θ_0) величину

$$y \cdot \frac{\theta^T x + \theta_0}{\|\theta\|}$$

Глядя на это выражение, что можем о ней сказать?

Отступ (margin)

Вспомним, что для гиперплоскости $\theta^T x + \theta_0 = 0$, расстояние (со знаком) от неё до точки x равно

$$\frac{\theta^T x + \theta_0}{\|\theta\|}.$$

Назовём **отступом (margin)** помеченной точки (x, y) по отношению к гиперплоскости (θ, θ_0) величину

$$y \cdot \frac{\theta^T x + \theta_0}{\|\theta\|}$$

Глядя на это выражение, что можем о ней сказать?

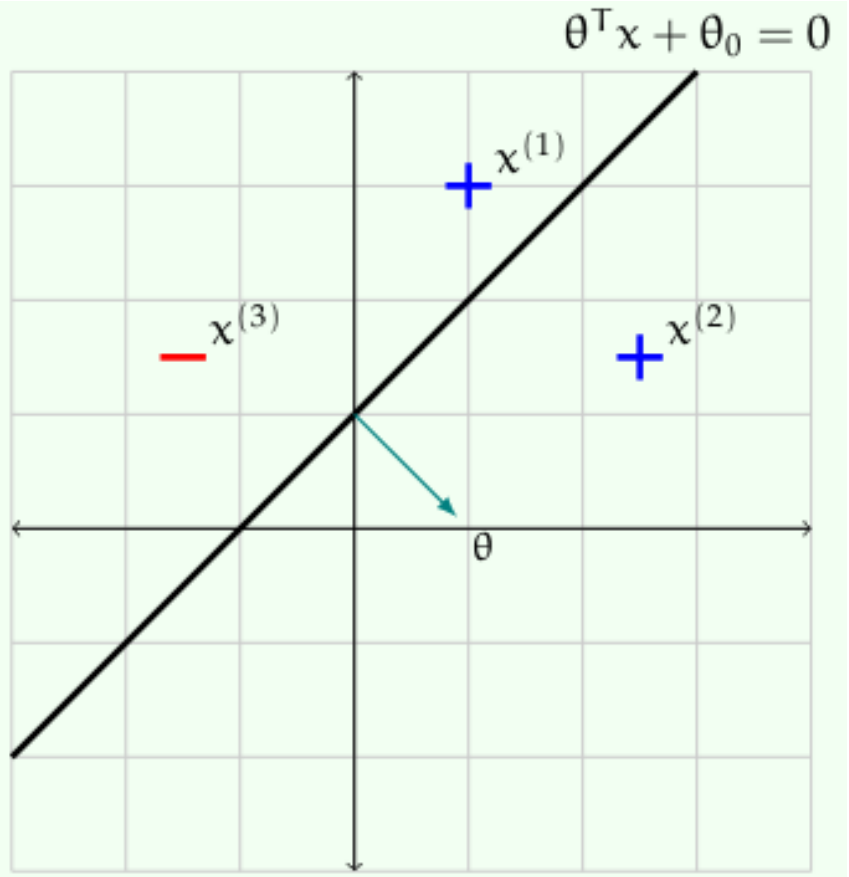
- Если точка правильно классифицирована, то отступ положительный. Иначе неотрицательный.
- Чем больше положительное значение отступа, тем «надежнее» она классифицирована.

Отступ (margin)

Назовём **отступом (margin) датасета** $\bar{\mathcal{D}}_n$ по отношению к гиперплоскости (θ, θ_0) наименьший отступ её точек:

$$\min_i \left(y \cdot \frac{\theta^T x + \theta_0}{\|\theta\|} \right)$$

Отступ (margin): Пример



$$\theta = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \ \theta_0 = 1.$$

$$y^{(1)} \cdot \frac{\theta^T x^{(1)} + \theta_0}{\|\theta\|} = -\frac{\sqrt{2}}{2}$$

$$y^{(2)} \cdot \frac{\theta^T x^{(2)} + \theta_0}{\|\theta\|} = \sqrt{2}$$

$$y^{(3)} \cdot \frac{\theta^T x^{(3)} + \theta_0}{\|\theta\|} = \sqrt{2}$$

Отступ данной выборки по отношению к (θ, θ_0) равна $-\frac{\sqrt{2}}{2}$.

Теорема о сходимости персептрона

Theorem (Сходимость персептрона (perceptron convergence))

Пусть выполняются эти условия:

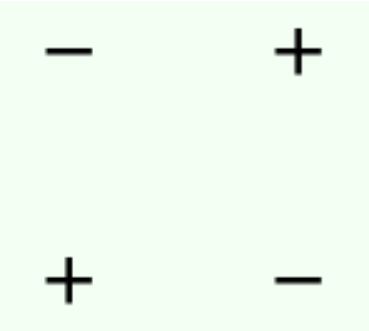
1. *существуют* θ^* *и скаляр* $\gamma > 0$ *такие, что* $y \cdot \frac{\theta^T x + \theta_0}{\|\theta\|} \geqslant \gamma$,
2. $\|x^{(i)}\| \leqslant R$ *для всех* $i = 1, \ldots, n$.

Тогда алгоритм персептрона совершит не более $(\frac{R}{\gamma})^2$ *ошибок. По истечении этого, алгоритм получит линейный классификатор, который правильно классифицирует весь датасет.*

Отсутствие линейной разделяемости

Если датасет *линейно разделяем*, то алгоритм персептрона очень хорошо работает и даёт нам хороший линейный классификатор.

Но условие линейной разделяемости очень ограничивающее! Классический пример датасета, который неразделяем:



Это называют **XOR dataset**.

Отсутствие линейной разделяемости

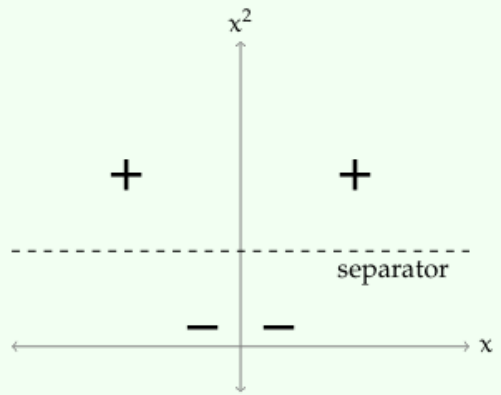
Рассмотрим одномерный пример линейно неразделяемого датасета:

Отсутствие линейной разделяемости

Рассмотрим одномерный пример линейно неразделяемого датасета:

Гениальная идея: перейдём на более высокую размерность!

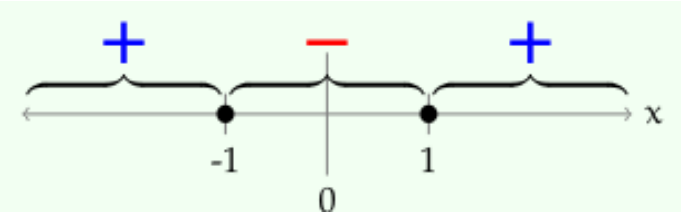
Трансформируем каждую точку x в двумерную точку по такой формуле:
$\phi(x) = [x, x^2]$



Отсутствие линейной разделяемости

В этой плоскости наши точки уже легко разделяются разделяющей прямой.

Например, это может быть раздяляющая прямая $x^2 - 1 > 0$. Используя найденный классификатор для 2D, мы можем вернуться в оригинальный 1D:



Представление признаков

Эта техника перевода датасета на более высокую размерность называется **представление признаков (feature representation)**.

Иными словами, признаки классифицируемых объектов представляются по-другому с целью решить сложную задачу классификации простыми способами в их новом представлении.

Это очень мощная идея, которая широко используется в ML.

Представление признаков: Полиномиальный базиз

Если признаки изучаемых объектов численные, то универсальный способ трансформировать их в высокие размерности — это **Полиномиальный базиз (polynomial basis)**.

Полиномиальный базиз k-го порядка — числовые признаки x представляется всеми возможными их произведениями до k-го порядка включительно.

Order	$d = 1$	in general
0	$[1]$	$[1]$
1	$[1, x]$	$[1, x_1, \ldots, x_d]$
2	$[1, x, x^2]$	$[1, x_1, \ldots, x_d, x_1^2, x_1x_2, \ldots]$
3	$[1, x, x^2, x^3]$	$[1, x_1, \ldots, x_1^2, x_1x_2, \ldots, x_1x_2x_3, \ldots]$
$\vdots$	$\vdots$	$\vdots$

Представление признаков: Полиномиальный базиз

Вернемся к XOR датасету.

В этом случае, трансформация выглядит так:

Представление признаков: Полиномиальный базиз

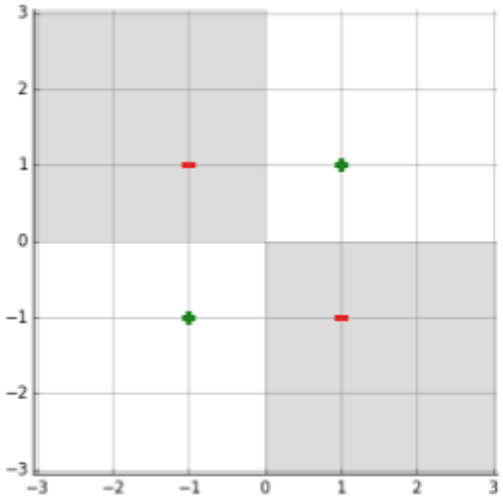
Вернемся к XOR датасету.
В этом случае, трансформация выглядит так:

$$\phi([x_1, x_2]) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$$

Персептрон найдет такую модель:
$\theta = (0, 0, 0, 0, 4, 0)$, $\theta_0 = 0$, соответствующий разделяющей гиперплоскости $4x_1x_2 > 0$
Как выглядит наш классификатор в оригинальной плоскости?

Представление признаков: Полиномиальный базиз

Вернемся к XOR датасету.
В этом случае, трансформация выглядит так:

$$\phi([x_1, x_2]) = [1, x_1, x_2, x_1^2, x_1 x_2, x_2^2]$$

Персептрон найдет такую модель:
$\theta = (0, 0, 0, 0, 4, 0)$, $\theta_0 = 0$, соответствующий разделяющей гиперплоскости $4x_1x_2 > 0$
Как выглядит наш классификатор в оригинальной плоскости?